Arrays, hashing + windows

Templates only. Named hooks stand for problem-specific conditions, state and output.

Hash map: complement lookup

```
Map<Long, Integer> seen = new HashMap<>();
int[] pair = {-1, -1};
for (int i = 0; i < a.length; i++) {
    long need = (long) target - a[i];
    if (seen.containsKey(need)) {
        pair = new int[]{seen.get(need), i}; break;
    }
    seen.put((long) a[i], i);
}
```

Query before insertion. pair contains two indices, or {-1,-1}.

Hashing: membership + frequency

```
Set<Integer> seen = new HashSet<>();
Map<Integer, Integer> count = new HashMap<>();
for (int x : a) {
    boolean firstTime = seen.add(x);
    count.merge(x, 1, Integer::sum);
}
```

seen stores processed values; count stores their multiplicity.

Prefix sums: inclusive range [l, r]

```
long[] p = new long[a.length + 1];
for (int i = 0; i < a.length; i++)
    p[i + 1] = p[i] + a[i];
long rangeSum = p[r + 1] - p[l];
```

p[i] covers the first i values. Valid with negative values.

Prefix sums: count ranges with sum k

```
Map<Long, Integer> count = new HashMap<>();
count.put(0L, 1);
long sum = 0, ans = 0;
for (int x : a) {
    sum += x;
    ans += count.getOrDefault(sum - k, 0);
    count.merge(sum, 1, Integer::sum);
}
```

Query earlier prefixes before storing the current prefix.

Two pointers: sorted pair scan

```
int l = 0, r = a.length - 1;
while (l < r) {
    long sum = (long) a[l] + a[r];
    if (sum == target) break;  // pair found
    if (sum < target) l++;
    else r--;
}
```

Ascending input; l < r after the loop means a pair was found.

Sliding window: fixed size

```
long sum = 0;
for (int i = 0; i < k; i++) sum += a[i];
long best = sum;
for (int r = k; r < a.length; r++) {
    sum += (long) a[r] - a[r-k];
    best = Math.max(best, sum);
}
```

1 <= k <= n. Add entering value, remove leaving value, record.

Sliding window: longest valid

```
int left = 0, best = 0;
for (int right = 0; right < n; right++) {
    add(a[right]);
    while (invalid()) remove(a[left++]);
    best = Math.max(best, right-left+1);
}
```

Hooks update window state. Empty window must be valid; prove monotonicity.

Sliding window: shortest qualifying

```
int left = 0, best = n + 1;
for (int right = 0; right < n; right++) {
    add(a[right]);
    while (left <= right && valid()) {
        best = Math.min(best, right-left+1);
        remove(a[left++]);
    }
}
// best == n+1 means no qualifying window.
```

Hooks: record and shrink while valid. Prove monotonicity first.

Search, sorted scans + rolling state

Array or answer-space search: monotonic predicate, opposite-truth endpoints, safe bounds.

Binary search: first true

```
// lo is false; hi is true (real or virtual).
while (lo + 1 < hi) {
    int mid = lo +
        (int) (((long) hi - lo) / 2);
    if (condition(mid)) hi = mid;
    else lo = mid;
}
// Answer boundary: hi.
```

For lower bound: lo=-1, hi=n, condition(i)=a[i]>=target.

Binary search: last true

```
// lo is true; hi is false (real or virtual).
while (lo + 1 < hi) {
    int mid = lo +
        (int) (((long) hi - lo) / 2);
    if (condition(mid)) lo = mid;
    else hi = mid;
}
// Answer boundary: lo.
```

For predecessor: lo=-1, hi=n, condition(i)=a[i]<=target.

Intervals: merge closed ranges

```
Arrays.sort(a,
    (x,y) -> Integer.compare(x[0], y[0]));
List<int[]> out = new ArrayList<>();
for (int[] x : a) {
    if (out.isEmpty()
            || out.get(out.size()-1)[1] < x[0])
        out.add(x.clone());
    else {
        int[] last = out.get(out.size()-1);
        last[1] = Math.max(last[1], x[1]);
    }
}
```

Sorts input order. Equal endpoints merge. Copy each output interval.

Greedy intervals: earliest finish

```
Arrays.sort(a,
    (x,y) -> Integer.compare(x[1], y[1]));
long end = Long.MIN_VALUE;
int kept = 0;
for (int[] x : a) {
    if (x[0] < end) continue;
    kept++; end = x[1];
}
// Minimum removals = a.length - kept.
```

Equal-value positive-length intervals. Touching endpoints are compatible.

Kadane: maximum nonempty sum

```
long cur = a[0], best = a[0];
for (int i = 1; i < a.length; i++) {
    cur = Math.max(a[i], cur + a[i]);
    best = Math.max(best, cur);
}
```

Nonempty input. cur ends at i; best is the best anywhere.

Rolling state: minimum + maximum product

```
long hi = a[0], lo = a[0], best = a[0];
for (int i = 1; i < a.length; i++) {
    long x = a[i], p = hi*x, q = lo*x;
    hi = Math.max(x, Math.max(p, q));
    lo = Math.min(x, Math.min(p, q));
    best = Math.max(best, hi);
}
```

Nonempty input; products must fit long. Preserve both old extrema.

Stack: nested / unresolved state

```
Deque<State> st = new ArrayDeque<>();
for (Token t : tokens) {
    if (opens(t)) st.push(context(t));
    else if (closes(t)) {
        if (st.isEmpty()) fail();
        else resolve(st.pop(), t);
    } else process(t);
}
// Check unresolved states if the task requires it.
```

Java-shaped hooks. Most recent unresolved context is processed first.

Monotonic deque: window maximum

```
Deque<Integer> q = new ArrayDeque<>();
int[] out = new int[a.length-k+1];
for (int r = 0; r < a.length; r++) {
    while (!q.isEmpty() && q.peekFirst() <= r-k)
        q.pollFirst();
    while (!q.isEmpty() && a[q.peekLast()] <= a[r])
        q.pollLast();
    q.offerLast(r);
    if (r >= k-1) out[r-k+1] = a[q.peekFirst()];
}
```

1 <= k <= n. Front expires; back removes dominated candidates.

BST: choose one branch

```
TreeNode node = root;
while (node != null && node.val != target)
    node = target < node.val
        ? node.left : node.right;
// node is the match, or null.
```

Requires BST ordering. Time follows height, not necessarily log n.

Monotonic stacks: the six guide formats

Attached guide numbering. Use fresh shared setup for each independent template.

Shared setup: indices + missing = -1

```
Deque<Integer> st = new ArrayDeque<>();
int[] next = new int[a.length];
int[] prev = new int[a.length];
Arrays.fill(next, -1);
Arrays.fill(prev, -1);
```

Fresh stack for each independent template. Keep only the needed arrays.

1. Next strictly greater

```
for (int i = 0; i < a.length; i++) {
    while (!st.isEmpty() && a[st.peek()] < a[i])
        next[st.pop()] = i;
    st.push(i);
}
```

Non-increasing values; resolve popped indices inside while.

2. Previous strictly greater

```
for (int i = 0; i < a.length; i++) {
    while (!st.isEmpty() && a[st.peek()] <= a[i])
        st.pop();
    if (!st.isEmpty()) prev[i] = st.peek();
    st.push(i);
}
```

Strictly decreasing values; read the surviving top after while.

3. Next >= and previous >

```
for (int i = 0; i < a.length; i++) {
    while (!st.isEmpty() && a[st.peek()] <= a[i])
        next[st.pop()] = i;
    if (!st.isEmpty()) prev[i] = st.peek();
    st.push(i);
}
```

Merged pass: next includes equals; previous is strict.

4. Next strictly smaller

```
for (int i = 0; i < a.length; i++) {
    while (!st.isEmpty() && a[st.peek()] > a[i])
        next[st.pop()] = i;
    st.push(i);
}
```

Non-decreasing values; resolve popped indices inside while.

5. Previous strictly smaller

```
for (int i = 0; i < a.length; i++) {
    while (!st.isEmpty() && a[st.peek()] >= a[i])
        st.pop();
    if (!st.isEmpty()) prev[i] = st.peek();
    st.push(i);
}
```

Strictly increasing values; read the surviving top after while.

6. Next <= and previous <

```
for (int i = 0; i < a.length; i++) {
    while (!st.isEmpty() && a[st.peek()] >= a[i])
        next[st.pop()] = i;
    if (!st.isEmpty()) prev[i] = st.peek();
    st.push(i);
}
```

Merged pass: next includes equals; previous is strict.

Operator + answer location

Boundary	Pop	Read answer
Next >	<	Inside while
Previous >	<=	After while
Next >= / previous >	<=	Both
Next <	>	Inside while
Previous <	>=	After while
Next <= / previous <	>=	Both

All scans go left to right. Stack values read bottom to top.
Each index is pushed and popped at most once: O(n).
Switching the merged operator flips which boundary includes equals.

Linked lists, trees + ordered structures

Node types are supplied by the task. Recursive depth uses the Java call stack.

Linked list: reverse links

```
ListNode prev = null, cur = head;
while (cur != null) {
    ListNode next = cur.next;
    cur.next = prev;
    prev = cur;
    cur = next;
}
head = prev;
```

Save next before rewiring. prev is the reversed prefix.

Linked list: fast / slow pointers

```
ListNode slow = head, fast = head;
boolean cycle = false;
while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow == fast) { cycle = true; break; }
}
```

Identity detects cycles. Without a cycle, slow approaches the midpoint.

Tree DFS: the three visit positions

```
void dfs(TreeNode node) {
    if (node == null) return;
    // visit(node);                 // preorder
    dfs(node.left);
    visit(node);                    // inorder
    dfs(node.right);
    // visit(node);                 // postorder
}
```

Hook: visit. Use exactly one visit position; shown form is inorder.

Recursion / tree DFS: return a result

```
Result solve(Node node) {
    if (base(node)) return baseResult(node);
    Result left = solve(node.left);
    Result right = solve(node.right);
    return combine(node, left, right);
}
```

Hooks: base, baseResult, combine. Define the call contract and decreasing input.

Heap: retain the largest k

```
PriorityQueue<Integer> h = new PriorityQueue<>();
for (int x : a) {
    h.offer(x);
    if (h.size() > k) h.poll();
}
// h.peek() is the kth largest, if at least k seen.
```

k >= 1. Reverse the comparator to retain the smallest k.

Tree BFS: freeze one level

```
Queue<TreeNode> q = new ArrayDeque<>();
if (root != null) q.offer(root);
while (!q.isEmpty()) {
    int size = q.size();
    while (size-- > 0) {
        TreeNode node = q.poll();
        visit(node);
        if (node.left != null) q.offer(node.left);
        if (node.right != null)
            q.offer(node.right);
    }
    finishLevel();
}
```

Hooks: visit, finishLevel. Freeze size before adding the next level.

BST: inherited lower / upper bounds

```
boolean valid(TreeNode node, long lo, long hi) {
    if (node == null) return true;
    if (node.val <= lo || node.val >= hi)
        return false;
    return valid(node.left, lo, node.val)
        && valid(node.right, node.val, hi);
}
```

Strict BST. Start with Long.MIN_VALUE / Long.MAX_VALUE; bounds come from ancestors.

Heaps: split + balance two halves

```
// low: max-heap; high: min-heap.
low.offer(x);
high.offer(low.poll());
if (high.size() > low.size())
    low.offer(high.poll());
double median = low.size() > high.size()
    ? low.peek()
    : ((long) low.peek() + high.peek()) / 2.0;
```

For each new x. low.size equals high.size or exceeds it by one.

Trie: shared-prefix insert + lookup

```
class Node {
    Node[] child = new Node[26]; boolean end;
}
Node root = new Node(), node = root; // insert
for (char ch : word.toCharArray()) {
    int c = ch - 'a';
    if (node.child[c] == null)
        node.child[c] = new Node();
    node = node.child[c];
}
node.end = true;
node = root;                          // lookup
for (char ch : prefix.toCharArray()) {
    node = node.child[ch - 'a'];
    if (node == null) break;
}
// Prefix exists: node != null.
// Exact word exists: node != null && node.end.
```

Lowercase a-z. Root is an empty Node. Adapt children for a different alphabet.

Backtracking: core families + parameter rules

Attached field-guide structure. Select state, choices, constraints and exact undo.

Backtracking: the six-step skeleton

```
void backtrack(STATE, CURRENT_PATH) {
    // 1. GOAL
    if (GOAL_REACHED) {
        save(copyOf(CURRENT_PATH)); return;
    }
    // 2. CHOICES
    for (CHOICE : AVAILABLE_CHOICES) {
        // 3. CONSTRAINTS
        if (INVALID_CHOICE) continue;
        // 4. CHOOSE
        makeChoice(CHOICE);
        // 5. EXPLORE
        backtrack(UPDATED_STATE, CURRENT_PATH);
        // 6. UNDO
        undoChoice(CHOICE);
    }
}
```

Pseudocode hooks. Subsets save every path without returning there.

Backtracking: index-forward / use once

```
out.add(new ArrayList<>(path)); // all subsets
for (int i = start; i < a.length; i++) {
    if (i > start && a[i] == a[i-1]) continue;
    path.add(a[i]);
    backtrack(i+1, path);
    path.remove(path.size()-1);
}
```

Hook: recursive call. Sort first for duplicates; remove skip for distinct input.

Backtracking: target + reuse allowed

```
if (remaining == 0) {
    out.add(new ArrayList<>(path)); return;
}
for (int i = start; i < a.length; i++) {
    if (a[i] > remaining) break;
    path.add(a[i]);
    backtrack(i, remaining-a[i], path);
    path.remove(path.size()-1);
}
```

Sorted distinct positive a. Use once: i+1; add same-level duplicate skip.

Backtracking: choose the next cut

```
if (start == s.length()) { save(path); return; }
for (int end = start; end < s.length(); end++) {
    if (!allowed(s, start, end)) continue;
    path.add(s.substring(start, end+1));
    backtrack(end+1, path);
    path.remove(path.size()-1);
}
```

Hooks: allowed, save (copy), backtrack. end is a choice; next start is end+1.

Choose loop + next recursive state

Family	Loop	Next state
Subset / use once	i = start..	i + 1
Combination / reuse	i = start..	i
Permutation	i = 0..	used[]
Position / row	legal choices	index + 1

Pass fixed input, progress, path, output.
Pass new scalar values; restore mutable path / used / board.
Save a copy. Skip equal siblings after sorting, not every equal value.

Backtracking: order matters / used[]

```
if (path.size() == a.length) {
    out.add(new ArrayList<>(path)); return;
}
for (int i = 0; i < a.length; i++) {
    if (used[i]) continue;
    if (i > 0 && a[i] == a[i-1] && !used[i-1])
        continue;
    used[i] = true; path.add(a[i]);
    backtrack(path, used);
    path.remove(path.size()-1); used[i] = false;
}
```

Sort first; used starts false. Remove duplicate rule when all values differ.

Backtracking: one choice per position

```
if (index == n) { save(path); return; }
for (Choice choice : choicesFor(index)) {
    if (!allowed(choice, path)) continue;
    choose(choice, path);
    backtrack(index+1, path);
    undo(choice, path);
}
```

Hooks define choices and constraints. Covers mapped letters and row placements.

Backtracking: path-local visited state

```
boolean search(State s) {
    if (invalid(s) || visited(s)) return false;
    if (goal(s)) return true;
    mark(s);
    boolean found = false;
    for (State next : neighbors(s)) {
        if (search(next)) { found = true; break; }
    }
    unmark(s);
    return found;
}
```

Java-shaped hooks. Restore marks on success and failure; do not use global visited.

Graphs, connectivity + interval sweeps

Global visited for components; path-local visited for backtracking. Mark on discovery.

BFS: shortest unweighted distance

```
int[] dist = new int[g.size()];
Arrays.fill(dist, -1);
Queue<Integer> q = new ArrayDeque<>();
dist[start] = 0; q.offer(start);
while (!q.isEmpty()) {
    int u = q.poll();
    for (int v : g.get(u)) {
        if (dist[v] != -1) continue;
        dist[v] = dist[u]+1; q.offer(v);
    }
}
```

List<List<Integer>> g, vertices 0..n-1. Multi-source: seed all sources at distance 0.

DFS: adjacency-list reachability

```
void dfs(int u, List<List<Integer>> g,
        boolean[] seen) {
    if (seen[u]) return;
    seen[u] = true;
    for (int v : g.get(u)) dfs(v, g, seen);
}
```

Global seen for components. Start from each unseen vertex to enumerate components.

Grid DFS: flood fill / component marking

```
void flood(int[][] b, int r, int c) {
    if (r < 0 || r >= b.length || c < 0
            || c >= b[0].length || b[r][c] != 1)
        return;
    b[r][c] = 0;
    flood(b,r+1,c); flood(b,r-1,c);
    flood(b,r,c+1); flood(b,r,c-1);
}
```

1 = land, 0 = water/visited. Mutates grid; use an explicit stack for large depth.

Intervals: before / overlap / after

```
List<int[]> out = new ArrayList<>();
int i = 0, lo = added[0], hi = added[1];
while (i < a.length && a[i][1] < lo)
    out.add(a[i++].clone());
while (i < a.length && a[i][0] <= hi) {
    lo = Math.min(lo, a[i][0]);
    hi = Math.max(hi, a[i][1]); i++;
}
out.add(new int[]{lo, hi});
while (i < a.length) out.add(a[i++].clone());
```

a contains sorted disjoint closed intervals. added is the new interval.

Topological sort: Kahn readiness queue

```
int[] degree = new int[g.size()];
for (List<Integer> edges : g)
    for (int v : edges) degree[v]++;
Queue<Integer> q = new ArrayDeque<>();
for (int u = 0; u < g.size(); u++)
    if (degree[u] == 0) q.offer(u);
List<Integer> order = new ArrayList<>();
while (!q.isEmpty()) {
    int u = q.poll(); order.add(u);
    for (int v : g.get(u))
        if (--degree[v] == 0) q.offer(v);
}
// order.size() < g.size() means a directed cycle.
```

Directed edge: prerequisite -> dependent. Consume ready vertices only.

Union-find: compression + union by size

```
class DSU {
    int[] p, size;
    DSU(int n) {
        p = new int[n]; size = new int[n];
        for (int i = 0; i < n; i++) {
            p[i] = i; size[i] = 1;
        }
    }
    int find(int x) {
        while (x != p[x]) {
            p[x] = p[p[x]]; x = p[x];
        }
        return x;
    }
    boolean union(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) {
            int t = a; a = b; b = t;
        }
        p[b] = a; size[a] += size[b];
        return true;
    }
}
```

Undirected, merge-only connectivity. Same group: find(a)==find(b).

Intervals: count simultaneous meetings

```
Arrays.sort(a,
    (x,y) -> Integer.compare(x[0], y[0]));
PriorityQueue<Integer> ends =
    new PriorityQueue<>();
int best = 0;
for (int[] x : a) {
    while (!ends.isEmpty() && ends.peek() <= x[0])
        ends.poll();
    ends.offer(x[1]);
    best = Math.max(best, ends.size());
}
```

Positive-duration meetings. Equality releases a room; merging loses concurrency.